

Homework set 1

Please **submit this Jupyter notebook through Canvas** no later than **Thursday November 7. Submit the notebook file with your answers (as .ipynb file) and a pdf printout. The pdf version can be used by the teachers to provide feedback. On canvas there are hints about creating a nice pdf version.**

Before you hand in, please make sure the notebook runs, by running "Restart kernel and run all cells..." from the Kernel menu.

Homework is in **groups of two**, and you are expected to hand in original work. Work that is copied from another group will not be accepted.

Exercise 0

Write down the names + student ID of the people in your group.

Format: Name [ID]

1. Kushnava Singha [15663388]
2. Shania Sinha [14379031]

Run the following cell to import NumPy and Pyplot.

```
In [4]: import numpy as np
import matplotlib.pyplot as plt
```

Writing efficient Numpy code

Colon (:) notation

In this section we give an introduction to indexing in Numpy. For further information see <https://numpy.org/doc/stable/user/basics.indexing.html>.

By using colon (:) notation you can manipulate entire subblocks of vectors and matrices without having to write a for loop. Let's define a vector first

```
In [3]: vec1 = np.array( [ 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120 ] )
```

Using square brackets we can, of course, access the individual elements of this array. For example, to access the fourth element (in human language) we write

```
In [4]: vec1[3]    # As you know, Python indexing starts at 0
```

Out [4]: 40

Frequently we want to manipulate a range of values of a vector, or a block of values of a matrix. For these scenarios, there is colon indexing. For example, to assign elements 2, 3, 4, 5 of `vec1` to a new vector `vec2`, we write

```
In [5]: # Get elements 2 to 5 from vec1 and put it in vec2
vec2 = vec1[2:6]
print(vec2)
```

[30 40 50 60]

This extends to matrices. For example, if we have a 5×5 matrix A and we want to change a subblock of this matrix this can be done as follows

```
In [ ]: # A 5x5 matrix of zeros
A = np.zeros( (5,5) )
print('Initial choice of A: \n', A)

# Change a 2x2 subblock of A
A[2:4, 2:4] = np.array([[1,2],[11,12]])
print('A after modifying a subblock: \n',A)
```

Initial choice of A:

```
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
```

A after modifying a subblock:

```
[[ 0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.]
 [ 0.  0.  1.  2.  0.]
 [ 0.  0. 11. 12.  0.]
 [ 0.  0.  0.  0.  0.]
```

There are many more possibilities with the Numpy colon notation, of which we mention a few:

```
In [7]: # a stepsize can be introduced using the notation start:stop:step
print('every second element: ', vec1[0:12:2])

# in the notation start:stop, omitting the start value means "start from
print('first three values: ', vec1[:3])

# negative numbers mean "count from the end"
print('last three values: ', vec1[-3:])

# a single colon means "all values"
print('all values: ', vec1[:])

# Example: the following changes the values in the existing array vec1
# It does not allocate a new array, like with vec1 = np.ones(...)
vec1[:] = 1.0

print('all values: ', vec1[:])
```

```

every second element: [ 10  30  50  70  90 110]
first three values:   [10 20 30]
last three values:    [100 110 120]
all values:           [ 10  20  30  40  50  60  70  80  90 100 110 120]
all values:           [1 1 1 1 1 1 1 1 1 1 1 1]

```

Performance advantage of vector operations

We just saw how to easily manipulate subblocks of vectors and matrices using colon indexing. This generally makes sense for short code. Without colon indexing, the one-line manipulations above would require a for loop and hence multiple lines of code.

Numpy also has many other convenient functions that replace for loops. For example, by using the function `numpy.sum` you can sum all the elements of a vector without writing a Python for loop. There are many more examples of this coding style, which is called vectorization (replacing for loops by vector commands).

Not only does it make for short code, vectorized code is often also much faster. This is because Python is in general a slow language, much slower than for example C. Each line incurs a certain overhead^[1]. Numpy operations are basically calls to C code that perform the vector manipulations efficiently. When executing a vector operation with colon notation, the Python overhead is incurred only once. On the other hand, when executing a Python loop, the Python overhead is incurred each time the code in the loop is executed. Therefore using vector operations with colon notation is generally preferred.

[^1]: A full discussion of this is outside the scope of this course.

Example: Matrix multiplication

Here we present a short demo showcasing *how much* faster vectorized operations actually are. We will write two versions of a function that computes a matrix product `C` of two matrices `A` and `B`. We will use the `%timeit` macro to investigate the performance of each version.

In the first version of our function we have three `for` loops. We iterate over each entry in matrix `C` and compute it using another `for` loop, which sums up all the products of corresponding entries in row `i` in matrix `A` and column `j` in matrix `B`.

Compute, on paper, the matrix product below and convince yourself that the following implementation is correct.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix} = ?$$

```
In [8]: A = np.array([[1,2,3],[4,5,6]], dtype=int)
        B = np.array([[10,11],[20,21],[30,31]], dtype=int)
```

```
In [9]: def slow_matmul(A, B, C):
        """
        Computes the matrix product of matrices A and B and stores it in matrix C
        """
        for i in range(A.shape[0]):
            for j in range(B.shape[1]):
                C[i, j] = 0
                for k in range(A.shape[1]):
                    C[i, j] += A[i,k] * B[k,j]
```

```
In [10]: C = np.zeros((A.shape[0], B.shape[1]), dtype=int)

        slow_matmul(A, B, C)
        print(C)
```

```
[[140 146]
 [320 335]]
```

In the second version we replace the third `for` loop with a vectorized `np.sum()`. In fact, we use two vectorized operations here. Notice how we efficiently compute all the products of corresponding entries `A` and `B`. By calling `A[i] * B[:, j]` we compute all the products at once, instead of doing it one-by-one as in the previous version. Look at the code and convince yourself that the implementation is correct.

```
In [11]: def faster_matmul(A, B, C):
        """
        Computes the matrix product of matrices A and B and stores it in matrix C
        """
        for i in range(A.shape[0]):
            for j in range(B.shape[1]):
                C[i, j] = np.sum(A[i] * B[:, j])
```

```
In [12]: A = np.array([[1,2,3],[4,5,6]], dtype=int)
        B = np.array([[10,11],[20,21],[30,31]], dtype=int)
        C = np.zeros((A.shape[0], B.shape[1]), dtype=int)

        faster_matmul(A, B, C)
        print(C)
```

```
[[140 146]
 [320 335]]
```

Now we will compare the performance of both versions by computing the matrix product of two 200×200 matrices. You can rerun the cells yourself, but keep in mind the execution might take a minute or two for the `slow_matmul` implementation.

```
In [13]: A = np.random.randint(0, 100, (200, 200), dtype=int)
        B = np.random.randint(0, 100, (200, 200), dtype=int)
```

```
In [14]: C = np.zeros((A.shape[0], B.shape[1]), dtype=int)

        %timeit slow_matmul(A, B, C)
```

7.13 s $\pm$ 946 ms per loop (mean $\pm$ std. dev. of 7 runs, 1 loop each)

```
In [15]: A = np.random.randint(0, 100, (200, 200), dtype=int)
B = np.random.randint(0, 100, (200, 200), dtype=int)
C = np.zeros((A.shape[0], B.shape[1]), dtype=int)

%timeit faster_matmul(A, B, C)
```

336 ms $\pm$ 3.87 ms per loop (mean $\pm$ std. dev. of 7 runs, 1 loop each)

On my machine `slow_matmul` ran on average 3.2 s $\pm$ 23 ms and `faster_matmul` ran on average 172 ms $\pm$ 4.42 ms, meaning that `faster_matmul` was more than 18 times faster.

Of course, you might already know this, but you should never write your own implementation of matrix multiplication (unless we specifically ask for it). The demo presented above was for illustrative purposes.

The most efficient way to multiply two matrices using Numpy is to use `@` operator which calls the `np.matmul` function.

```
In [15]: %timeit A @ B
```

8.79 ms $\pm$ 471 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)

On my machine `A @ B` needed, on average, 9.9 ms $\pm$ 151 μ s to execute meaning that it is more than 323 times faster than `slow_matmul` and 17 times faster than `faster_matmul`.

Exercise 1

Suppose x_1 and x_2 are two real numbers and that approximate values $\hat{x}_1$ and $\hat{x}_2$ are available such that the relative errors satisfy $\frac{|\Delta x_1|}{|x_1|} \leq r_1$ and $\frac{|\Delta x_2|}{|x_2|} \leq r_2$ for two real numbers r_1 and r_2 . What is the bound for the relative error in the product $\hat{x}_1 \hat{x}_2$?

Answer:

Let

$$\Delta x_1 = \hat{x}_1 - x_1 \quad \text{and} \quad \Delta x_2 = \hat{x}_2 - x_2$$

Then:

$$\hat{x}_1 = x_1 + \Delta x_1 \quad \text{and} \quad \hat{x}_2 = x_2 + \Delta x_2$$

The approximate product $\hat{x}_1 \hat{x}_2$ can be written as:

$$\hat{x}_1 \hat{x}_2 = (x_1 + \Delta x_1)(x_2 + \Delta x_2)$$

$$\hat{x}_1 \hat{x}_2 = x_1 x_2 + x_1 \Delta x_2 + x_2 \Delta x_1 + \Delta x_1 \Delta x_2$$

The error in the product $\hat{x}_1 \hat{x}_2$ is:

$$\Delta(x_1x_2) = \hat{x}_1\hat{x}_2 - x_1x_2 = x_1\Delta x_2 + x_2\Delta x_1 + \Delta x_1\Delta x_2$$

The relative error in $\hat{x}_1\hat{x}_2$ is then given by

$$\frac{|\Delta(x_1x_2)|}{|x_1x_2|} = \frac{|x_1\Delta x_2 + x_2\Delta x_1 + \Delta x_1\Delta x_2|}{|x_1x_2|}$$

Using the triangle inequality, we can bound this by

$$\frac{|\Delta(x_1x_2)|}{|x_1x_2|} \leq \frac{|x_1\Delta x_2|}{|x_1x_2|} + \frac{|x_2\Delta x_1|}{|x_1x_2|} + \frac{|\Delta x_1\Delta x_2|}{|x_1x_2|}$$

Simplifying each term, we get:

$$\frac{|x_1\Delta x_2|}{|x_1x_2|} = \frac{|\Delta x_2|}{|x_2|} \leq r_2$$

$$\frac{|x_2\Delta x_1|}{|x_1x_2|} = \frac{|\Delta x_1|}{|x_1|} \leq r_1$$

$$\frac{|\Delta x_1\Delta x_2|}{|x_1x_2|} = \frac{|\Delta x_1|}{|x_1|} \cdot \frac{|\Delta x_2|}{|x_2|} \leq r_1r_2$$

Summing these, we get

$$\frac{|\Delta(x_1x_2)|}{|x_1x_2|} \leq r_1 + r_2 + r_1r_2$$

Conclusion: Hence, the bound for the relative error in the product $\hat{x}_1\hat{x}_2$ is:

$$\frac{|\Delta(x_1x_2)|}{|x_1x_2|} \leq r_1 + r_2 + r_1r_2$$

Exercise 2

Solve computational exercise 2.3 from Heath (page 99) using your own implementation of LU decomposition with partial pivoting and of back- and forward substitution. Also read carefully the derivation of the matrix for use in the next exercise.

As you noticed, there is a short section on colon (:) notation and vectorization in Numpy at the start of this homework set. In the LU decomposition there are a few places where for loops can easily be replaced by vectorized commands. You are asked to use vectorized operations and colon notation in these cases. The same holds for forward and backward substitution.

Please first implement the functions for which the headers are in the next cell. Then apply these to solve the problem at hand.

```

In [184... # Answer:
def factorize(A):
    """
    Returns a triple (P, L, U) describing the LU decomposition with parti
    """
    n = A.shape[0]          # number of rows in A
    L = np.eye(n)           # initialize L as the identity matrix with lower
    P = np.eye(n)           # initialize P as the identity matrix to store ro
    U = np.copy(A)          # initialize U as a copy of A, overwritten with u

    for k in range(n):
        # Partial pivoting
        maxRow = np.argmax(abs(U[k:, k])) + k # Find index of max elemen

        # Swap rows in U and record in P and L if necessary
        if maxRow != k:
            U[[k, maxRow], :] = U[[maxRow, k], :] # Swap rows in U
            P[[k, maxRow], :] = P[[maxRow, k], :] # Store the swap in
            L[[k, maxRow], :k] = L[[maxRow, k], :k] # Swap rows in L on

        # Vectorized update of L and U for rows below the pivot row k
        L[k+1:, k] = U[k+1:, k] / U[k, k] # Update L column-w
        U[k+1:, k:] -= np.outer(L[k+1:, k], U[k, k:]) # Update rows of U

    return P, L, U

def forwardSubstitution(L, P, b):
    """
    Computes the solution to the linear system Ly = Pb.
    """
    n = L.shape[0]
    y = np.zeros(n)
    rhs = P @ b
    for i in range(n):
        y[i] = (rhs[i] - np.dot(L[i, :i], y[:i]))

    return y

def backwardSubstitution(U, y):
    """
    Computes the solution to the linear system Ux = y
    """
    n = U.shape[0]
    x = np.zeros(n)
    for i in range(n-1, -1, -1):
        x[i] = (y[i] - np.dot(U[i, i+1:], x[i+1:])) / U[i, i]
        x[i] = np.where(np.abs(x[i]) < 1e-10, 0, x[i])

    return x

```

```

In [185... # Forces on the joints based on the equations in Heath ex 2.3
# Note: angle between links is 45 deg, sin(45) = cos(45) = sqrt(2)/2
A = np.array([[ 0, -1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0],
               [ 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
               [-1/np.sqrt(2), 0, 0, 1, 1/np.sqrt(2), 0, 0, 0, 0, 0, 0, 0, 0],
               [-1/np.sqrt(2), 0, -1, 0, -1/np.sqrt(2), 0, 0, 0, 0, 0, 0, 0, 0],
               [ 0, 0, 0, -1, 0, 0, 0, 1, 0, 0, 0, 0, 0],

```

```

[ 0, 0, 0, 0, 0, 0, -1, 0, 0, 0, 0, 0, 0],
[ 0, 0, 0, 0, -1/np.sqrt(2), -1, 0, 0, 1/np.sqrt(2), 1, 0,
[ 0, 0, 0, 0, 1/np.sqrt(2), 0, 1, 0, 1/np.sqrt(2), 0, 0, 0,
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, -1, 0, 0, 1],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, -1, -1/np.sqrt(2), 0, 0, 1/np.sqrt(2),
[ 0, 0, 0, 0, 0, 0, 0, 0, -1/np.sqrt(2), 0, -1, -1/np.sqrt(2),
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, -1/np.sqrt(2), -1]]

# External forces acting on the joints
b = np.array([0, 10, 0, 0, 0, 0, 0, 15, 0, 20, 0, 0, 0])

```

In [186... *# Compute the LU decomposition of A (PA = LU)*
P,L,U = factorize(A)

```

# Solve the linear system Ax = b
# First solve Ly = Pb
y = forwardSubstitution(L,P,b)

# Then solve Ux = y
x = backwardSubstitution(U,y)
print(x)

```

```

[-28.28427125  20.          10.          -30.          14.14213562
 20.           0.          -30.           7.07106781  25.
 20.          -35.35533906  25.           ]

```

Exercise 3

In exercise 2, you were asked to solve exercise 2.3 from Heath. In this assignment you are asked to create a program that automatically creates the equations for the forces in the truss.

We first need a description of a truss. A simple way of doing this is by giving the coordinates of the joints and then list the pairs of joints that are connected, as follows:

In [187... *# coordinates of joints*
joints1 = [(0,0), (1,0), (1,1), (2,1), (2,0), (3,0), (3,1), (4,0)]

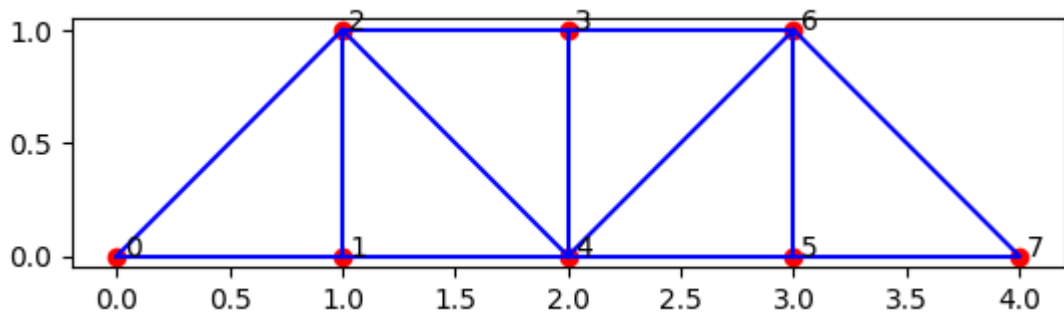
pairs indicating which joints are connected
members1 = [(0,2), (0,1), (1,2), (2,3), (2,4), (1,4),
(3,4), (3,6), (4,6), (4,5), (5,6), (6,7), (5,7)]

To check this input, we plot it

In [189... **def** DrawTruss(joints, members):
xx = [p[0] **for** p **in** joints]
yy = [p[1] **for** p **in** joints]
plt.scatter(xx,yy,color='r')
for (j,p) **in** enumerate(joints):
plt.text(p[0]+0.03,p[1],f"{j}")
for (jA, jB) **in** members:
xxMember = np.array([joints[i][0] **for** i **in** (jA,jB)])


```
yyMember = np.array( [joints[i][1] for i in (jA,jB) ] )
plt.plot(xxMember, yyMember, 'b')
```

```
In [190... (fig, ax) = plt.subplots()
DrawTruss(joints1, members1)
ax.set_aspect(1.0)
```



(a)

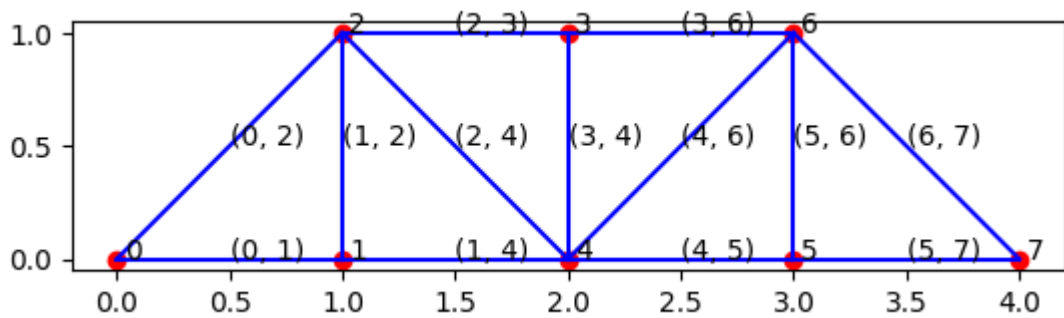
Modify `DrawTruss` such that the routine can draw the numbers of the members in the graph halfway each member.

```
In [191... def DrawTrussMarked(joints, members):
    """
    Draws a truss with joints marked by numbers and members by numbers in

    Inputs:
    - joints: a list of (x, y) coordinates for each joint
    - members: a list of tuples indicating which joints are connected by

    Outputs:
    - A plot showing the truss structure with connecting joints marked
    """
    xx = [ p[0] for p in joints ]
    yy = [ p[1] for p in joints ]
    # plot the joints
    plt.scatter(xx,yy,color='r')
    # label the joints
    for (j,p) in enumerate(joints):
        plt.text(p[0]+0.03,p[1],f"{j}")
    # plot the lines
    for (jA, jB) in members:
        xxMember = np.array( [joints[i][0] for i in (jA,jB) ] )
        yyMember = np.array( [joints[i][1] for i in (jA,jB) ] )
        plt.plot(xxMember, yyMember, 'b')
    # label the lines
    for j,(jA, jB) in enumerate(members):
        plt.text(0.5*(joints[jA][0]+joints[jB][0]), 0.5*(joints[jA][1]+joints[jB][1]), f"{j}")
```

```
In [192... (fig, ax) = plt.subplots()
# Draw the truss with midpoint labels
DrawTrussMarked(joints1, members1)
ax.set_aspect(1.0)
```



(b)

Study how in exercise 2.3 the equations for the forces are derived. Then write a routine to compute the corresponding matrix automatically from the information above, so arrays like `joints1` and `members1`.

You may have noticed that joint 1 and the y component of joint 8 are not included in the equations of exercise 2.3. This is because joint 1 is rigidly fixed and joint 8 is fixed vertically. It is recommended that you first include all joints, so also joint 1 and the y component of joint 8 in the truss of exercise 2.3. In a separate step, you can select the matrix rows that are "free" (not fixed in one or both of the directions).

Create a vector with external forces (the downward arrows at joints 2, 5, 6 in exercise 2.3), and solve the system.

Write the resulting force values in the graph of the truss near the appropriate members.

The forces in a member can be compressive (like they "push" the member to become shorter) or tension forces (like they pull the member to become longer). Which member or members experience the largest compressive force? Which one experience the largest tension force?

(In general one would try to keep such maximum forces as low as possible since that would allow one to use less material.)

```
In [193... def calculateTrussForce(joints, members, fixed_joints, external_forces):
    """
    Analyzes a truss structure and calculates the member forces using the
    Inputs:
    - joints: a list of (x, y) coordinates for each joint
    - members: a list of tuples indicating which joints are connected by
    - fixed_joints: a list of indices of fixed joints
    - external_forces: a dictionary with joint indices as keys and extern

    Returns:
    - forces: a list of member forces
    """
    num_joints = len(joints)
    num_members = len(members)

    # Create the matrix A representing the equilibrium equations
```

```

A = np.zeros((2 * num_joints, num_members))    # 2 equations per joi

for member_index, (j1, j2) in enumerate(members):
    # Coordinates of the two joints connected by this member
    x1, y1 = joints[j1]
    x2, y2 = joints[j2]

    # Calculate the length and components of the member force
    length = np.sqrt((x2 - x1)**2 + (y2 - y1)**2)
    cos_theta = (x2 - x1) / length
    sin_theta = (y2 - y1) / length

    # Each member affects two joints, j1 (starting point) and j2 (end
    A[2 * j1, member_index] = cos_theta        # Horizontal componen
    A[2 * j1 + 1, member_index] = sin_theta    # Vertical component

    A[2 * j2, member_index] = -cos_theta       # Horizontal componen
    A[2 * j2 + 1, member_index] = -sin_theta  # Vertical component

# Remove rows corresponding to fixed joints to get the reduced matrix
A_reduced = np.delete(A, fixed_joints, axis=0)

# Set b vector to account for external forces at given joint
b = np.zeros(2 * num_joints)
for joint, force in external_forces.items():
    b[2 * joint + 1] = force

# Remove rows corresponding to fixed joints from b vector
b_reduced = np.delete(b, fixed_joints, axis=0)

# Solve Ax = b for the reduced system
P, L, U = factorize(A_reduced)
y = forwardSubstitution(L, P, b_reduced)
x = backwardSubstitution(U, y)

return x

```

In [194...

```

# Inputs
fixed_joints = [0, 1, 15]
external_forces = {1: 10, 4: 15, 5: 20}

# Calculate the forces in the truss members
forces = calculateTrussForce(joints1, members1, fixed_joints, external_forces)
print(f"Forces:\n{forces}\n")

# Print the forces for each member
for i, force in enumerate(forces):
    if force > 0:
        print(f"Member {i+1} is in tension with force {force:.2f}")
    else:
        print(f"Member {i+1} is in compression with force {abs(force):.2f}")

```

Forces:

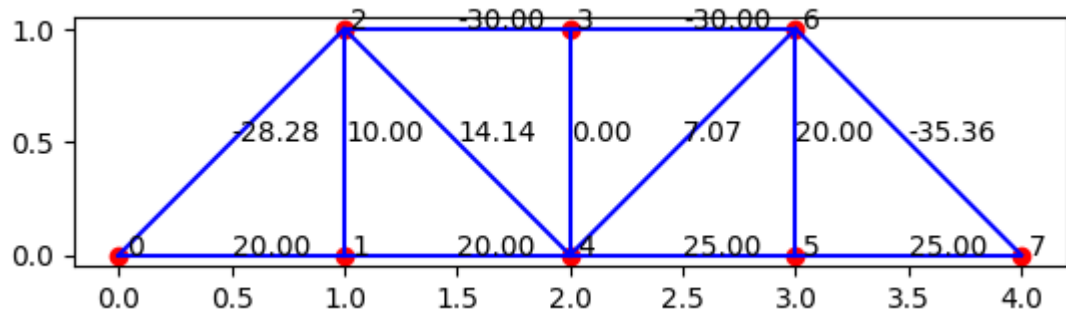
```
[ -28.28427125  20.          10.          -30.          14.14213562
   20.           0.         -30.           7.07106781  25.
   20.        -35.35533906  25.          ]
```

Member 1 is in compression with force 28.28
 Member 2 is in tension with force 20.00
 Member 3 is in tension with force 10.00
 Member 4 is in compression with force 30.00
 Member 5 is in tension with force 14.14
 Member 6 is in tension with force 20.00
 Member 7 is in compression with force 0.00
 Member 8 is in compression with force 30.00
 Member 9 is in tension with force 7.07
 Member 10 is in tension with force 25.00
 Member 11 is in tension with force 20.00
 Member 12 is in compression with force 35.36
 Member 13 is in tension with force 25.00

```
In [195... def DrawTrussForces(joints, members, forces):
    """
    A function to draw a truss structure with forces on the members.
    Inputs:
    - joints: a list of (x, y) coordinates for each joint
    - members: a list of tuples indicating which joints are connected by
    - forces: a list of forces for each member

    Outputs:
    - A plot showing the truss structure with forces labeled on each memb
    """
    xx = [ p[0] for p in joints ]
    yy = [ p[1] for p in joints ]
    # plot the joints
    plt.scatter(xx,yy,color='r')
    # label the joints
    for (j,p) in enumerate(joints):
        plt.text(p[0]+0.03,p[1],f"{j}")
    # plot the lines
    for (jA, jB) in members:
        xxMember = np.array( [joints[i][0] for i in (jA,jB) ] )
        yyMember = np.array( [joints[i][1] for i in (jA,jB) ] )
        plt.plot(xxMember, yyMember, 'b')
    # label the lines
    for j,(jA, jB) in enumerate(members):
        force_value = forces[j]
        plt.text(0.5*(joints[jA][0]+joints[jB][0]), 0.5*(joints[jA][1]+jo
```

```
In [196... (fig, ax) = plt.subplots()
# Draw the truss with forces
DrawTrussForces(joints1, members1, forces)
ax.set_aspect(1.0)
```



```
In [197... # Find the members with the largest tension and compression forces
max_tension = max(forces)
max_compression = min(forces)

print(f"\nLargest tension force is {max_tension:.2f}")
print(f"Largest compression force is {abs(max_compression):.2f}")
```

Largest tension force is 25.00
Largest compression force is 35.36